

Briefing do Case — PIX Compliance Swarm

Desafio Sênior de IA — Verity Candidato: Gabriel Carvalho Mendes Prazo: 4 dias Metodologia: Spec-Driven Development (SDD)

1. Contexto do desafio

Um squad de compliance financeiro precisa monitorar continuamente os normativos PIX publicados pelo Banco Central do Brasil. A solução pedida é um **enxame de 5 a 7 agentes Pydantic AI** que colete, extraia, analise, indexe e consolide esses normativos, expondo os resultados via relatório estruturado, API consultável e base vetorizada (RAG).

O documento do desafio determina explicitamente que **todos os dados devem ser tratados como fictícios**, com fixtures de no mínimo 50 normativos mock. Isso libera o projeto de depender do site real do BCB e é a alavanca mais importante de redução de escopo em 4 dias.

1.1 O que a banca declara que vai avaliar

Eixo	O que significa na prática
Design do agente Pydantic AI	Uso correto de <code>Agent</code> , ferramentas tipadas, <code>RunContext</code> , padrões de orquestração (sequencial, paralelo, loop com condição)
Modelos Pydantic	Tipagem rigorosa, validadores customizados, coerência de schema entre entrada, ferramentas e saída
Integração Bedrock	Configuração do provider AWS, tratamento de credenciais, seleção/fallback de modelos
Ferramentas e MCP	Servidor MCP funcional com transporte SSE e consumo correto pelo agente
API FastAPI	Swagger, tratamento de erros, modelos alinhados com o agente
Execução	Pipeline de ponta a ponta funcionável no ambiente descrito
Engenharia	Organização do repo, commits legíveis, testes, scripts reproduzíveis, containerização eficiente
Guardrail	Deteção e mascaramento de PII antes de invocar o LLM ou persistir
Arquitetura	Separação clara: agente vs. ferramenta vs. API vs. MCP vs. guardrail, com decisões documentadas

1.2 Requisito transversal explícito

O desafio pede uma seção **“Desenvolvimento e ferramentas”** no README, declarando forma de desenvolvimento, recursos consultados, métodos de orquestração e metodologia spec-driven aplicada. O uso de IA assistida é permitido — o que não é aceito é entregar código que não se compreende. O fato de o projeto inteiro ser conduzido por SDD com specs versionadas **é**, por si só, a evidência mais forte que se pode dar nessa seção.

2. Estratégia de execução em 4 dias

2.1 Princípios que governam o recorte

- O caminho crítico é o pipeline ponta a ponta.** Um fluxo completo e demonstrável vale mais do que sete agentes sofisticados que não se conectam. Fecha-se a espinha dorsal primeiro; profundidade depois.
- Tudo que é AWS roda local por padrão, exceto o LLM.** MinIO no lugar de S3 real, pgvector no lugar de OpenSearch Serverless, APScheduler no lugar de EventBridge — com o código de infra AWS documentado e o adaptador pronto. O avaliador precisa conseguir subir o projeto com `docker compose up` sem ter uma conta AWS.
- Bedrock é o caminho padrão de execução, não uma opção entre outras.** O desafio pede Bedrock nominalmente e o critério de avaliação nomeia essa integração explicitamente — tratá-la como intercambiável com um double é o risco mais alto do projeto. `LLM_PROVIDER=bedrock` é o valor padrão do `.env.example` e o caminho documentado no README. Existe um *test double* offline, mas ele é ferramenta de teste, não modo de operação — ver ADR-05.
- Contrato antes de comportamento.** Os modelos Pydantic são congelados cedo (SPEC-002). Todo agente subsequente programa contra esses tipos, o que permite paralelizar a implementação sem retrabalho.
- Evidência é entregável, não subproduto.** Logs, screenshots e vídeo são itens avaliados. São coletados durante o desenvolvimento, não reconstruídos na última hora.
- Abstração só onde há uma segunda implementação real ou um teste que precise dela.** SOLID aplicado por reflexo é, na prática, uma violação de KISS e de YAGNI — e um avaliador sênior lê excesso de `Protocol` /interface sem uso concreto como insegurança, não como rigor. Toda interface deste projeto precisa responder “qual é a segunda implementação, ou qual teste substitui esta dependência?”. Sem resposta, usa-se a classe concreta.

2.2 Decisões de arquitetura (ADRs resumidas)

#	Decisão	Alternativa descartada	Justificativa
ADR-01	pgvector concreto (Postgres), sem interface <code>VectorStore</code>	OpenSearch Serverless, ou abstração com stub das duas opções	O desafio aceita qualquer um dos dois; construir uma segunda implementação apenas para “provar” seria YAGNI puro. Documenta-se a escolha e o caminho de migração em prosa, não em código morto.
ADR-02	<code>ObjectStore</code> como protocolo, com implementação MinIO	Classe concreta única	Aqui a interface se justifica de verdade: a mesma classe serve S3 real trocando <code>endpoint_url</code> por variável de ambiente — é uma segunda implementação latente e de custo zero, não especulação.
ADR-03	APScheduler local + IaC do EventBridge documentado, dentro do próprio Orchestrator	Spec e módulo de agendamento separados	Agendamento é ~60 linhas sobre o mesmo endpoint do pipeline; separar em uma spec própria era segmentação sem ganho — fundido na spec do Orchestrator.
ADR-04	Site mock do BCB servido em container	Scraping do bcb.gov.br	O desafio manda tratar dados como fictícios. Scraping determinístico é testável e não depende de rede.
ADR-05	Bedrock como provider padrão; <i>test double</i> offline exclusivo para <code>tests/</code> , sem fallback silencioso	Provider “fake” plugável e intercambiável em runtime	Falta de credencial deve falhar alto e claro , nunca degradar silenciosamente para o <i>double</i> — isso evita que o avaliador rode o projeto achando que usou Bedrock sem de fato usar. O <i>double</i> existe porque o próprio desafio pede “fallback ou seleção de modelos” na integração — é a resposta a esse item, não uma fuga dele.
ADR-06	Guardrail como camada obrigatória no boundary do LLM	Checagem ad-hoc por agente	Ponto único de aplicação, impossível de contornar por esquecimento, e testável isoladamente.
ADR-07	Orchestrator como Harness explícito, incluindo o agendamento	Agente delegando a agentes por tool calling; agendamento em módulo separado	Controle de fluxo determinístico, retry e observabilidade num único lugar coerente. Delegação via tool fica demonstrada em um trecho específico dentro dele.

2.3 Distribuição por dia

Dia	Foco	Specs	Marco
1	Fundação e contratos	001 → 006	Modelos congelados, fixtures prontas, guardrail testado, storage no ar
2	Coleta e compreensão	007 → 011	MCP SSE funcional; scraping → extração → análise → gap analysis rodando
3	Conhecimento e exposição	012 → 015	RAG indexado, API com Swagger, relatório JSON+PDF, orquestrador (com agendamento) fechando o ciclo
4	Empacotamento e prova	016 → 019	Compose subindo tudo, testes verdes, README, diagramas, vídeo, evidências

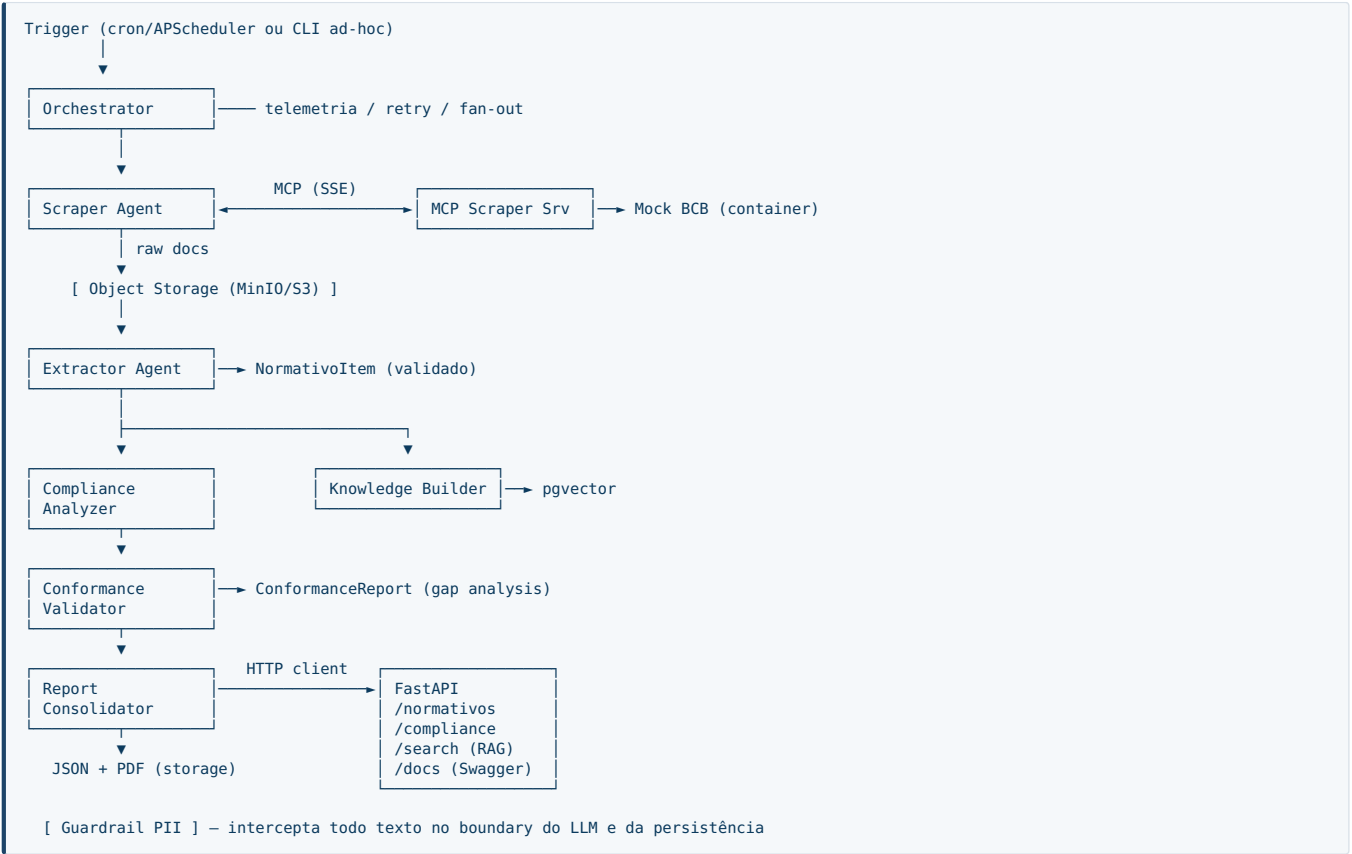
Regra de corte: se o Dia 3 terminar com o pipeline incompleto, os bônus (SPEC-019) são abandonados sem hesitação e o Dia 4 é integralmente usado para fechar o ciclo e produzir evidência. Um pipeline completo e bem documentado supera um pipeline parcial com alertas SNS.

3. Arquitetura alvo

3.1 Enxame de agentes

Agente	Responsabilidade	Skill	Entrada → Saída
Orchestrator	Coordena o pipeline, agenda execuções, distribui tarefas, aplica retry e consolida telemetria	<code>orchestrator-skill</code>	<code>PipelineRequest</code> → <code>PipelineResult</code>
Scraper	Coleta normativos, detecta novos/alterados por hash, persiste bruto no object storage	<code>scraper-skill</code> (MCP/SSE)	<code>ScrapeRequest</code> → <code>ScrapeResult</code>
Extractor	Extrai texto de PDF/HTML e estrutura em modelo validado	<code>extractor-skill</code>	<code>RawDocument</code> → <code>NormativoItem</code>
Compliance Analyzer	Categoriza regras em participantes, tarifas, liquidação, segurança, SLA, interoperabilidade	<code>compliance-analyzer-skill</code>	<code>NormativoItem</code> → <code>list[RegraExtraida]</code>
Conformance Validator	Compara versões, detecta deltas, produz gap analysis	<code>conformance-validator-skill</code>	<code>list[RegraExtraida]</code> → <code>ConformanceReport</code>
Knowledge Builder	Faz chunking, gera embeddings e mantém o índice vetorial	<code>knowledge-builder-skill</code>	<code>NormativoItem</code> → <code>IndexResult</code>
Report Consolidator	Gera relatório JSON e PDF; publica resultados via cliente HTTP na API	<code>report-consolidator-skill</code>	<code>ConformanceReport</code> → <code>ReportOutput</code>

3.2 Fluxo de ponta a ponta



O Analyzer e o Knowledge Builder rodam em paralelo (`asyncio.gather`), o que satisfaz o critério de “padrões de orquestração paralelos” com uma justificativa real: não há dependência de dados entre eles.

3.3 Estrutura de repositório

```

pix-compliance-swarm/
├── CLAUDE.md                # contrato persistente do Claude Code
├── README.md                # entregável avaliado
├── pyproject.toml
├── requirements.txt
├── .env.example
├── Makefile
├── docker-compose.yml
├── specs/                   # SDD – uma spec por arquivo
│   ├── SPEC-001-foundation.md
│   ├── SPEC-002-domain-models.md
│   └── ...
├── skills/                  # entregável avaliado
│   ├── orchestrator-skill/SKILL.md
│   ├── scraper-skill/SKILL.md
│   └── ...
├── docs/
│   ├── architecture.md
│   ├── diagrams/
│   ├── spec-methodology.md
│   └── evidence/           # logs, screenshots, links de vídeo
├── src/pix_compliance/
│   ├── config.py           # pydantic-settings
│   ├── models/             # SPEC-002
│   ├── guardrails/         # SPEC-004
│   ├── providers/          # SPEC-005 – bedrock_provider.py (padrão)
│   ├── storage/            # SPEC-006 – object_store.py (protocolo) / vector_store.py (concreto, pgvector)
│   ├── tools/              # ferramentas tipadas compartilhadas
│   ├── agents/             # um módulo por agente
│   ├── orchestration/      # SPEC-015 (inclui agendamento)
│   ├── api/                # SPEC-013
│   └── cli.py
├── mcp_servers/
│   └── scraper_sse/         # SPEC-007
├── fixtures/
│   ├── normativos.json     # >= 50 mock
│   └── documents/          # >= 3 PDF/HTML mock
├── mock_bcb/               # site fictício servido em container
├── tests/
│   └── doubles/            # SPEC-005 – offline_provider.py (test double, não faz parte de src/)
└── docker/

```

4. Convenções do SDD neste projeto

4.1 Anatomia de um arquivo de spec

Todo arquivo em `specs/` segue o mesmo esqueleto. A uniformidade é o que permite ao Claude Code executar uma spec sem contexto adicional:

```

# SPEC-0XX – <título>

- **Status:** draft | in-progress | done
- **Dia:** N
- **Depende de:** SPEC-0YY, SPEC-0ZZ
- **Requisito do desafio:** <trecho literal que esta spec satisfaz>

## Objetivo
Uma frase. O que passa a ser verdade quando esta spec fecha.

## Escopo
### Dentro
- ...
### Fora (explicitamente)
- ...

## Contratos
Assinaturas, modelos e interfaces que esta spec cria ou consome.

## Entregáveis
Lista de caminhos de arquivo.

## Critérios de aceite
- [ ] Verificáveis por comando ou teste, nunca subjetivos.

## Notas de implementação
Armadilhas conhecidas, decisões já tomadas, o que não tentar.

```

4.2 Regras de trabalho

- Nenhuma linha de código antes da spec correspondente estar escrita.** Se durante a implementação surgir uma decisão não prevista, ela volta para a spec antes de virar código.
- Critério de aceite é comando.** “Funciona bem” não é aceite; `pytest tests/test_guardrails.py -q` passando é.
- Um commit por spec, no mínimo.** Mensagem no formato `feat(spec-007): servidor MCP SSE do scraper`. Isso produz o histórico legível que o desafio cobra.
- A spec é atualizada ao fechar.** Status vira `done` e desvios da versão original ficam registrados na seção de notas. O diff das specs é a narrativa do projeto.
- Escopo negativo é obrigatório.** A seção “Fora” existe para conter a expansão de escopo — o inimigo número um de um prazo

de 4 dias.

4.3 Princípios de código: aplicados, não decorados

SOLID, YAGNI e KISS orientam este projeto, mas na direção oposta à intuição comum de “usar o máximo de padrão possível para parecer sênior”. A leitura que se busca é a inversa: um avaliador experiente reconhece senioridade em **abstração justificada**, não em abstração abundante.

Regras concretas adotadas neste projeto:

- **SRP por agente.** Cada agente do enxame tem uma responsabilidade e um contrato de entrada/saída — nunca dois agentes fazendo a mesma coisa por caminhos diferentes.
- **DIP e ISP só com seam real.** Interface (`Protocol`) se justifica quando existe uma segunda implementação de verdade (`ObjectStore` : MinIO e S3 real via a mesma classe) ou quando um teste precisa substituir a dependência (provider LLM, para rodar suíte sem rede). Sem uma das duas razões, usa-se a classe concreta — é o caso do `VectorStore` , que fica concreto sobre pgvector (ADR-01).
- **OCP no guardrail.** Novo detector de PII se adiciona sem tocar em nenhum agente — é o único ponto do projeto onde extensibilidade futura paga o custo de abstração hoje, porque a lista de padrões de PII plausivelmente cresce.
- **YAGNI contra interface especulativa.** Nenhuma classe abstrata sem uso concreto no próprio repositório. Onde o desafio permite duas opções (ex.: OpenSearch ou pgvector) e apenas uma é implementada, a outra fica documentada em prosa no README — não como stub de código morto.
- **KISS no agendamento.** Fundido no Orchestrator (SPEC-015) em vez de módulo e spec próprios — o volume de código não justificava a segmentação.

Comentários humanizados. O objetivo do comentário é responder a uma pergunta que um leitor atento faria — nunca parafrasear a linha de código. Convenção adotada:

- Identificadores (variáveis, funções, classes) em **inglês**; vocabulário de domínio do BCB/PIX mantido como está — `normativo` , `inciso` , `regra` , `vigencia` não se traduzem, são termos técnicos do setor.
- Docstrings e comentários de linha em **português**.
- Todo comentário justifica uma decisão não óbvia: por que este algoritmo e não o mais simples, por que esta ordem de operações, por que este caso de borda é tratado assim. Comentário que apenas descreve o que a linha seguinte já diz é removido na revisão.
- Módulos com lógica de negócio não trivial (guardrail, conformance validator, chunking do RAG) recebem uma docstring de módulo ou classe explicando o raciocínio de domínio antes do código, não apenas o “o quê”.

Esta convenção entra no `CLAUDE.md` do projeto como regra de geração de código, não como sugestão de estilo.

5. Catálogo de specs

Dia 1 — Fundação e contratos

SPEC-001 — Fundação do projeto e configuração

Depende de: —

Objetivo. Repositório executável com dependências resolvidas, configuração tipada e observabilidade mínima.

Dentro: `virtualenv` e `pyproject.toml/requirements.txt`; `src/pix_compliance/config.py` com `pydantic-settings` lendo todas as variáveis de ambiente (credenciais AWS, região, IDs de modelo Bedrock, URL da API, DSN do Postgres, endpoint do object storage); `.env.example` completo e comentado; logging estruturado em JSON com `correlation_id` por execução; `Makefile` com alvos `install` , `run` , `test` , `lint` , `up` , `down` ; esqueleto do `pytest` e do `ruff` .

Fora: qualquer lógica de agente; Docker (fica na SPEC-016).

Aceite: - `make install` conclui em ambiente limpo - `python -c "from pix_compliance.config import settings; print(settings.model_dump())"` imprime a configuração sem lançar exceção - Nenhum segredo hardcoded — `grep -rn "AKIA" src/` retorna vazio - `make lint` sem erros

Notas. `Settings` deve falhar rápido e com mensagem clara quando faltar variável obrigatória. Esse é o primeiro contato do avaliador com o projeto: uma mensagem tipo “falta AWS_REGION; copie .env.example para .env” vale mais do que um traceback.

SPEC-002 — Modelos de domínio (Pydantic v2)

Depende de: SPEC-001

Objetivo. Congelar o vocabulário de tipos do sistema inteiro. É a spec mais importante do projeto — todo agente subsequente programa contra ela.

Dentro:

Modelo	Campos centrais
NormativoItem	id, titulo, tipo (enum: Resolução BCB, Instrução Normativa, Circular, Comunicado), numero, artigo, inciso, texto, data_publicacao, data_vigencia, categoria, url_origem, hash_conteudo, versao
RegraExtraida	regra_id, normativo_id, categoria (enum das 6 categorias), enunciado, obrigatoriedade (enum), prazo, atores_afetados, confianca
ConformanceReport	report_id, gerado_em, itens: list[ConformanceItem], resumo, criticidade_maxima
ConformanceItem	regra_id, status (enum: conforme, não conforme, novo, alterado, revogado), delta, recomendacao, severidade
SearchQuery / SearchResult	query, top_k, filtros; resultado com score, trecho, normativo_id
ReportOutput	json_path, pdf_path, total_normativos, total_regras, total_gaps, gerado_em
PipelineRequest / PipelineResult	entrada e saída do orquestrador
RawDocument	source_uri, content_type, bytes_ref, hash_conteudo, coletado_em

Validadores exigidos (o desafio cita `field_validator` e `model_validator` nominalmente): - `data_vigencia` não pode ser anterior a `data_publicacao` — `model_validator(mode="after")` - `hash_conteudo` deve ser SHA-256 hex de 64 caracteres — `field_validator - texto` não vazio após `strip()`, com normalização de espaços - `confianca` e `score` em `[0.0, 1.0]` via `Annotated[float, Field(ge=0, le=1)]` - `categoria` restrita ao enum, com coerção case-insensitive de string - `numero` de normativo validado por regex de formato

Fora: persistência; qualquer chamada de LLM.

Aceite: - `pytest tests/test_models.py -q` verde, cobrindo caminho feliz e cada validador rejeitando entrada inválida - Schemas JSON exportados em `docs/schemas/` via `model_json_schema()` - Todos os modelos com `model_config = ConfigDict(extra="forbid")`

Notas. Usar `StrEnum` para as categorias. Definir os modelos com `frozen=True` onde fizer sentido semântico. Este arquivo será citado no README como entregável nº 2 do desafio.

SPEC-003 — Fixtures e corpus mock

Depende de: SPEC-002

Objetivo. Produzir o universo de dados fictícios exigido explicitamente pelo desafio.

Dentro: gerador determinístico (`seed` fixo) de **≥ 50 normativos** em `fixtures/normativos.json` com `id`, `titulo`, `tipo`, `data`, `categoria`, `resumo`, `status`; **≥ 3 documentos** completos em PDF e HTML em `fixtures/documents/`, com estrutura realista de artigos e incisos; **pelo menos um documento contendo PII plantada** (CPF e CNPJ fictícios) para exercitar o guardrail; **pelo menos dois pares de versões** do mesmo normativo, com diferença conhecida, para exercitar o gap analysis; site mock do BCB em `mock_bcb/` — HTML estático com página de listagem e links para os documentos.

Fora: o servidor MCP que consome esse site (SPEC-007).

Aceite: - `python -m fixtures.generate` regenera tudo de forma idempotente - `jq 'length' fixtures/normativos.json ≥ 50` - Todo item das fixtures valida contra `NormativoItem` - Site mock servido em `python -m http.server` responde na página de listagem

Notas. Os pares de versões precisam ter delta *conhecido e documentado* — é isso que torna o resultado do Conformance Validator verificável no vídeo de evidência em vez de apenas plausível. Documentar os deltas esperados em `fixtures/EXPECTED_DELTAS.md`.

SPEC-004 — Camada de guardrail e PII

Depende de: SPEC-002

Objetivo. Garantir que nenhum dado sensível chegue ao LLM ou ao armazenamento sem mascaramento.

Dentro: detectores de CPF, CNPJ, e-mail, telefone e chave PIX aleatória, com **validação de dígito verificador** para CPF/CNPJ (reduz falso positivo drasticamente); política de mascaramento preservando formato (`123.***.***-01`); `PIIReport` com tipo, posição e contagem por ocorrência; ponto único de aplicação — um wrapper `guard(text) -> GuardedText` que **todo** provider LLM e **toda** escrita de storage obrigatoriamente atravessa; verificação de tamanho e de injeção de prompt básica (delimitadores, instruções embutidas).

Fora: anonimização reversível; criptografia.

Aceite: - `pytest tests/test_guardrails.py -q` verde, incluindo CPFs válidos, inválidos e sequências numéricas que não são CPF - Teste de integração provando que o provider LLM **não pode** ser invocado sem passar pelo guardrail - Log estruturado registra toda detecção com contagem, nunca com o valor original

Notas. Validar dígito verificador é o detalhe que separa uma implementação sênior de um regex ingênuo — e é barato. Mencionar explicitamente no README.

SPEC-005 — Provider LLM e embeddings (Bedrock)

Depende de: SPEC-001, SPEC-004

Objetivo. Integração real com o Amazon Bedrock como caminho padrão de execução, com um test double isolado para a suíte offline — nunca o inverso.

Dentro: cliente `bedrock-runtime` via `boto3` com credenciais exclusivamente por variável de ambiente; provider de chat compatível com Pydantic AI apontando para um modelo Claude no Bedrock; provider de embeddings com Titan; **cadeia de fallback** de modelos configurável por lista de IDs — na falha de um, tenta o próximo, com backoff exponencial; `LLM_PROVIDER=bedrock` como **valor padrão** de `config.py` e de `.env.example`; ausência de credencial ou de acesso ao modelo falha alto, com mensagem acionável ("Credenciais Bedrock ausentes ou modelo sem acesso liberado no console. Configure AWS_ACCESS_KEY_ID/SECRET, ou use LLM_PROVIDER=offline apenas para rodar a suíte de testes.") — **nunca** degrada silenciosamente para outro provider; tratamento tipado de `ThrottlingException`, `ValidationException`, `AccessDeniedException`; `OfflineProvider` determinístico em `tests/doubles/`, fora de `src/`, selecionável só por `LLM_PROVIDER=offline` e usado exclusivamente pela suíte de testes.

Fora: fine-tuning; batch inference.

Aceite: - Com `LLM_PROVIDER=bedrock` e sem credencial no ambiente, a aplicação recusa subir com a mensagem acima — nunca cai para outro provider por conta própria - `LLM_PROVIDER=offline` `pytest -q` roda a suíte inteira sem rede — usado apenas em `tests/` - Teste de fallback com o primeiro modelo mockado para falhar demonstra a troca de modelo dentro do próprio Bedrock - Documentado no README qual IAM policy mínima é necessária, e o lembrete de que acesso a modelo no Bedrock precisa ser solicitado por modelo no console antes de qualquer teste manual

Notas. Este é o ponto de maior risco de avaliação do projeto: se o double for intercambiável em runtime com o Bedrock, o avaliador não tem como saber se a integração real funciona, e uma queda silenciosa para o double seria lida como não cumprimento do requisito. Solicitar acesso aos modelos no console do Bedrock **no primeiro dia**, não perto da entrega — a aprovação de alguns modelos não é instantânea. O vídeo de evidência precisa mostrar uma invocação real ao Bedrock, com `model_id` e consumo de tokens no log.

SPEC-006 — Camada de armazenamento

Depende de: SPEC-001, SPEC-004

Objetivo. Object storage com abstração real (há duas implementações de fato) e vector store concreto (há apenas uma), sem interface especulativa.

Dentro: `ObjectStore` como `Protocol`, com implementação `boto3` apontando para MinIO via `endpoint_url` — a mesma classe serve S3 real trocando a variável de ambiente, o que torna a interface uma abstração paga por um seam real, não hipotético; `PgVectorStore` como **classe concreta**, sem protocolo — schema de tabela, índice HNSW ou IVFFlat, `upsert` e `similarity_search` com filtro por metadados; migrations simples em SQL versionado.

Fora: provisionamento AWS; replicação; qualquer abstração ou stub de OpenSearch — a escolha por pgvector (ADR-01) e o caminho de migração ficam documentados em prosa no `docs/architecture.md`, não em código.

Aceite: - Teste de round-trip: upload de bytes no object store, download, hash idêntico - Teste de round-trip vetorial: upsert de 10 vetores, busca retorna o esperado por similaridade - `docker compose up postgres minio` sobe ambos e os testes passam contra eles - Nenhuma classe abstrata ou protocolo sem implementação concreta no repositório

Notas. Definir a dimensionalidade do embedding a partir do modelo Titan escolhido e travá-la em `config.py`. Incompatibilidade de dimensão é o erro mais comum e mais chato de diagnosticar tarde.

Dia 2 — Coleta e compreensão

SPEC-007 — Servidor MCP do Scraper (transporte SSE)

Depende de: SPEC-003, SPEC-006

Objetivo. Expor a coleta de normativos como servidor MCP com transporte SSE — requisito nominal do desafio.

Dentro: servidor MCP em `mcp_servers/scraper_sse/` expondo as ferramentas `list_normativos(filtros)`, `fetch_normativo(id)` e `detect_changes(since)`; transporte SSE, servido em porta configurável; toda ferramenta com schema de entrada e saída tipado; coleta contra o site mock via `httpx` + parser HTML; detecção de novo/alterado por **hash SHA-256 do conteúdo** comparado ao último hash conhecido; persistência do bruto no `ObjectStore`; documentação de integração em `mcp_servers/scraper_sse/README.md` com o bloco de configuração pronto para copiar.

Fora: o agente que consome (SPEC-008).

Aceite: - Servidor sobe e responde handshake MCP via SSE - Cliente MCP externo lista as três ferramentas com seus schemas - `detect_changes` retorna vazio na segunda execução consecutiva sem mudança, e retorna o item alterado após modificação do fixture - Documentação de integração testada por um terceiro seguindo apenas o README

Notas. Este é um dos itens mais visíveis do desafio, porque é nominalmente citado três vezes no enunciado. Vale gravar um trecho específico do vídeo mostrando o handshake SSE e a listagem de ferramentas.

SPEC-008 — Scraper Agent

Depende de: SPEC-005, SPEC-007

Objetivo. Agente Pydantic AI que consome o servidor MCP e decide o que coletar.

Dentro: `Agent` com `deps_type` carregando o cliente MCP e o object store via `RunContext`; conexão ao servidor MCP SSE como toolset; `output_type=ScrapeResult`; política de retry com backoff para falha de transporte; `skills/scraper-skill/SKILL.md`.

Fora: extração de conteúdo.

Aceite: - Execução via CLI coleta os documentos do mock e devolve `ScrapeResult` validado - Servidor MCP derrubado durante a execução produz erro claro e tipado, não traceback cru - `SKILL.md` descrevendo responsabilidade, ferramentas, input e output

SPEC-009 — Extractor Agent

Depende de: SPEC-002, SPEC-005

Objetivo. Converter documentos brutos em `NormativoItem` validados.

Dentro: extração de PDF (`pdfplumber`) e HTML (`selectolax` ou `BeautifulSoup`) como **ferramentas tipadas determinísticas**, não como trabalho do LLM; o LLM entra apenas para estruturar campos ambíguos (artigo, inciso, data de vigência) com `output_type=NormativoItem`; guardrail aplicado ao texto **antes** da chamada; reparo de validação — em falha de validação Pydantic, uma segunda tentativa devolve ao modelo a mensagem de erro (loop com condição de parada, máximo de 2 tentativas); `skills/extractor-skill/SKILL.md`.

Fora: categorização de regras.

Aceite: - Os 3 documentos mock produzem `NormativoItem` válidos - PDF corrompido gera erro tratado, não quebra o pipeline - Teste comprovando que o loop de reparo é acionado e limitado a 2 tentativas

Notas. O loop de reparo é a evidência direta do critério “loops com condições” da avaliação. Instrumentar com log explícito.

SPEC-010 — Compliance Analyzer Agent

Depende de: SPEC-009

Objetivo. Categorizar regras nas seis dimensões pedidas pelo desafio.

Dentro: `Agent` com `output_type=list[RegraExtraida]`; system prompt com definição operacional de cada categoria — participantes, tarifas, liquidação, segurança, SLA, interoperabilidade; processamento em lote com concorrência limitada por semáforo; score de confiança por regra, com marcação para revisão humana abaixo de um limiar configurável; `skills/compliance-analyzer-skill/SKILL.md`.

Fora: comparação entre versões.

Aceite: - Cada uma das 6 categorias exercitada por ao menos um fixture - Regras de baixa confiança sinalizadas na saída - Processamento concorrente respeitando o limite configurado

SPEC-011 — Conformance Validator Agent

Depende de: SPEC-010

Objetivo. Produzir o gap analysis — comparar versões e classificar deltas.

Dentro: diff semântico entre conjuntos de `RegraExtraida` de versões diferentes do mesmo normativo; classificação em novo, alterado, revogado, inalterado; `delta` legível descrevendo a mudança; `recomendacao` acionável e `severidade`; saída `ConformanceReport`; `skills/conformance-validator-skill/SKILL.md`.

Fora: geração de relatório em PDF.

Aceite: - Os pares de versões da SPEC-003 produzem exatamente os deltas documentados em `EXPECTED_DELTAS.md` - Comparação com versão inexistente trata como coleção inicial, sem erro - `pytest tests/test_conformance.py -q` verde

Notas. Casar a saída com os deltas esperados transforma este agente em demonstração objetiva. É o momento mais forte do vídeo.

Dia 3 — Conhecimento e exposição

SPEC-012 — Knowledge Builder Agent (RAG)

Depende de: SPEC-006, SPEC-005

Objetivo. Indexar normativos em embeddings e servir busca semântica.

Dentro: chunking consciente de estrutura — quebra por artigo/inciso em vez de janela fixa, preservando `normativo_id`, `artigo` e `categoria` como metadados; embeddings via Titan em lote; upsert idempotente por `chunk_id` determinístico; `search(SearchQuery) -> list[SearchResult]` com filtro por metadados; `skills/knowledge-builder-skill/SKILL.md`.

Fora: reranking; busca híbrida (registrar como evolução futura no README).

Aceite: - Reindexação do mesmo corpus não duplica chunks - Consulta semântica sobre um termo presente em um único normativo retorna aquele normativo no topo - Filtro por categoria restringe corretamente o conjunto de resultados

Notas. Chunking por artigo é uma decisão de domínio, não técnica — normativos são estruturados por natureza e ignorar isso destrói precisão. Vale um parágrafo no README.

SPEC-013 — API FastAPI

Depende de: SPEC-002, SPEC-006, SPEC-012

Objetivo. Serviço HTTP documentado, com os endpoints nominalmente exigidos.

Dentro: `GET /normativos` com paginação e filtros por tipo, categoria e período; `GET /compliance` retornando análises e gap analysis, com filtro por severidade; `GET /search?query=...&top_k=` executando RAG; `GET /health` com checagem de dependências; `POST /runs` disparando execução ad-hoc do pipeline; `response_model` em todo endpoint reusando os modelos da SPEC-002; exception handlers devolvendo erro estruturado com `correlation_id`; metadados de OpenAPI preenchidos — título, descrição, versão, tags e exemplos por endpoint; Swagger em `/docs`.

Fora: autenticação (registrar como fora de escopo consciente no README).

Aceite: - `/docs` renderiza com descrições e exemplos em todos os endpoints - `pytest tests/test_api.py -q` cobrindo 200, 404 e 422 em cada rota - 422 de validação retorna corpo estruturado, não o default cru do FastAPI

Notas. O desafio pede screenshot do Swagger como evidência. Um `/docs` com exemplos preenchidos e descrições reais é desproporcionalmente barato de fazer e visualmente convincente.

SPEC-014 — Report Consolidator Agent

Depende de: SPEC-011, SPEC-013

Objetivo. Gerar o relatório final e cumprir o requisito de invocar a API FastAPI como cliente HTTP.

Dentro: relatório JSON conforme `ReportOutput`; relatório PDF via `reportlab` com capa, sumário executivo, tabela de normativos coletados, regras por categoria e seção de gap analysis com severidade; **cliente HTTP** publicando o resultado na API, com URL lida de variável de ambiente (requisito literal da seção 2 do desafio); upload de ambos os artefatos no object store; `skills/report-consolidator-skill/SKILL.md`.

Fora: templates customizáveis; envio por e-mail.

Aceite: - JSON e PDF gerados a partir do corpus completo - API indisponível gera degradação controlada — relatório persiste localmente e o erro é logado, sem perder o trabalho - URL da API exclusivamente via `settings`, sem literal no código

Notas. Este agente é o que fecha o requisito “invocar uma API FastAPI como cliente HTTP para ação final” da tarefa principal. Deixar isso explícito no SKILL.md e no README, porque é um item de checklist do avaliador.

SPEC-015 — Orchestrator Agent (Harness) e agendamento

Depende de: SPEC-008 a SPEC-014

Objetivo. Coordenar o enxame, demonstrar os padrões de orquestração avaliados e disparar a execução periódica — tudo sobre o mesmo endpoint (fundido com o antigo escopo de agendamento por KISS: o volume de código de scheduling não justificava spec e módulo próprios).

Dentro:

Orquestração: três padrões, cada um com justificativa de domínio documentada — **sequencial** (scrape → extract), **paralelo** via `asyncio.gather` (analyzer e knowledge builder, que não dependem um do outro), **loop com condição** (retry por agente e o reparo de validação da SPEC-009); `RunContext` com dependências compartilhadas — providers, stores, cliente HTTP, `correlation_id`; política de falha por etapa: fatal, degradável ou ignorável; `PipelineResult` agregando status, duração e contagens por etapa; delegação agente-para-agente demonstrada em ao menos um ponto, via ferramenta; `skills/orchestrator-skill/SKILL.md`.

Agendamento: `APScheduler` com cron configurável por variável de ambiente, chamando o mesmo handler usado pelo disparo ad-hoc via CLI; **snippet de IaC do EventBridge** (Terraform ou CDK) em `docs/aws/`, com a regra de schedule e o target apontando para o mesmo endpoint — EventBridge e APScheduler nunca divergem de handler; lock simples impedindo execuções sobrepostas.

Fora: execução distribuída; filas; deploy real na AWS.

Aceite: - `make run` executa o pipeline completo de ponta a ponta e imprime `PipelineResult` - Falha injetada em etapa degradável não aborta o pipeline; falha em etapa fatal aborta com mensagem clara - Log completo com `correlation_id` correlacionando todas as etapas de uma execução - Métrica de duração por etapa presente na saída - Scheduler com intervalo de 1 minuto executa e loga duas execuções consecutivas - Duas execuções simultâneas: a segunda é rejeitada pelo lock - Snippet de EventBridge revisado e comentado

Notas. O log desta execução é literalmente um dos entregáveis pedidos (“logs completos mostrando scraping → extração → análise → consolidação”). Salvar em `docs/evidence/pipeline-run.log`.

Dia 4 — Empacotamento e prova

SPEC-016 — Containerização

Depende de: SPEC-013, SPEC-007, SPEC-015

Objetivo. `docker compose up` sobe o sistema inteiro sem passo manual.

Dentro: Dockerfiles multi-stage para API, servidor MCP e scheduler, rodando com usuário não-root; `docker-compose.yml` com os serviços `api`, `mcp-scraper`, `scheduler`, `postgres` (pgvector), `minio`, `mock-bcb`; healthchecks e `depends_on` com condição; volumes para persistência; script de bootstrap criando bucket e aplicando migrations na primeira subida; `.dockerignore`.

Fora: Kubernetes; registry.

Aceite: - `docker compose up -d` a partir de repositório limpo deixa todos os serviços saudáveis - `/docs` acessível no host e o servidor MCP respondendo - `docker compose down -v && docker compose up -d` reproduz o estado sem intervenção manual - Imagens em tamanho razoável, com camadas de dependência cacheadas

Notas. “Eficiência de containerização” é critério explícito. Multi-stage e usuário não-root são baratos e sinalizam senioridade.

SPEC-017 — Testes e observabilidade

Depende de: todas as anteriores

Objetivo. Suíte que roda offline e telemetria que torna o pipeline auditável.

Dentro: testes unitários de modelos, guardrails e ferramentas; testes de integração de storage e API com containers; um teste ponta a ponta com `LLM_PROVIDER=offline` (o test double da SPEC-005, restrito a `tests/`) cobrindo todo o pipeline; fixtures de `pytest` compartilhadas; logging estruturado com `correlation_id` propagado por `RunContext`; contadores por etapa — documentos coletados, regras extraídas, gaps encontrados, tokens consumidos, latência; workflow de CI em GitHub Actions rodando lint e testes.

Fora: cobertura exaustiva; testes de carga.

Aceite: - `make test` verde sem rede e sem credenciais AWS - Teste ponta a ponta executando o pipeline inteiro - CI verde no repositório - Cobertura reportada, com foco declarado em modelos e guardrails

SPEC-018 — Documentação, diagramas, skills e evidências

Depende de: todas as anteriores

Objetivo. Produzir os entregáveis documentais, que no critério de avaliação pesam tanto quanto o código.

Dentro: - **README** com: descrição da solução, diagramas, dependências, instalação e variáveis de ambiente, execução (scraping, análise, API), subida via Docker, referência às skills, metodologia de especificação, integração com servidores MCP e a seção “Desenvolvimento e ferramentas” - **Diagramas:** contêiner e componente em Mermaid, no estilo C4 — enxame, fluxo de dados e integrações AWS - **7 arquivos SKILL.md**, um por agente, em formato uniforme - `docs/spec-methodology.md`: como o SDD foi aplicado, por que specs numeradas com escopo negativo, como o `CLAUDE.md` e o Claude Code participaram do fluxo - `docs/evidence/`: log completo do pipeline, screenshots do Swagger e de cada etapa, exemplos de request/response dos três endpoints - **Vídeo** demonstrando scraping, análise, geração de relatório e consulta à API, com narração das etapas

Aceite: - Um terceiro consegue subir o projeto seguindo apenas o README - Todos os 11 entregáveis da seção 5 do desafio mapeados e localizáveis - Diagramas renderizam no GitHub - Vídeo cobrindo as quatro etapas pedidas

Notas. Reservar tempo real para isto — 3 a 4 horas, não os 20 minutos finais. É aqui que o trabalho fica visível. O roteiro do vídeo deve ser escrito antes da gravação.

SPEC-019 — Bônus (executar apenas se as anteriores estiverem fechadas)

Depende de: SPEC-015 (Orchestrator)

Dentro: publicação em SNS/SQS quando novo normativo é detectado ou gap crítico é encontrado, com LocalStack no compose; seção no README sobre **onde** LangChain, Dify e n8n se encaixariam na arquitetura, com justificativa de tradeoff — não uso decorativo, mas análise de quando cada um agregaria; nota comparando MCP, Harness e Skills como padrões de composição.

Aceite: - Mensagem publicada e consumida no LocalStack, com evidência em log - Seção de integrações escrita como análise de arquitetura, não como lista de buzzwords

Notas. A seção analítica sobre Dify/n8n/LangChain custa 40 minutos e endereça um bônus explícito do desafio com muito mais impacto do que uma integração superficial. Se o tempo for curto, fazer só ela e abandonar o SNS/SQS.

6. Rastreabilidade — requisito do desafio → spec

Requisito	Spec
Entrada estruturada em modelo Pydantic	002, 015
Ferramentas tipadas com modelos de I/O	002, 008-014
Ao menos uma ferramenta via MCP com transporte SSE	007, 008
Consulta a base de dados ou índice	006, 012
Invocar API FastAPI como cliente HTTP, URL por env var	013, 014
Retorno validado em modelo Pydantic	002, 015
Enxame de 5-7 agentes	008-015 (7 agentes)
Validadores <code>field_validator</code> / <code>model_validator</code>	002
Bedrock LLM + Titan Embeddings, credenciais por env, como caminho padrão	005, 012
S3 para documentos brutos e relatórios	006, 014
OpenSearch Serverless ou pgvector	006 (pgvector concreto, OpenSearch documentado em prosa)
EventBridge para agendamento	015
Endpoints <code>/normativos</code> , <code>/compliance</code> , <code>/search</code> + Swagger	013
Guardrail de PII antes do LLM	004
Docker e docker-compose	016
SKILL.md por agente	008-015, 018
Metodologia spec-driven documentada	018 e este documento
Relatório JSON + PDF	014
Base vetorizada acessível	012, 013
Fixtures $\geq$ 50 normativos	003
$\geq$ 3 documentos PDF/HTML mock	003
Logs, screenshots e vídeo	015, 017, 018
Diagrama de arquitetura	018
Seção “Desenvolvimento e ferramentas”	018
Código sem abstração especulativa (SOLID/YAGNI/KISS aplicados)	002, 005, 006, 015 e seção 4.3
Comentários humanizados (por quê, não o quê)	seção 4.3, aplicado em toda spec de código
Bônus: SQS/SNS, LangChain/Dify/n8n	019

7. Riscos e mitigações

Risco	Probabilidade	Mitigação
MCP com SSE consumir tempo desproporcional	Alta	Atacar na manhã do Dia 2, com o servidor isolado e testado antes de acoplar ao agente
Incompatibilidade de dimensão de embedding	Média	Travar a dimensão em <code>config.py</code> na SPEC-006 e validar no upsert
Extração de PDF produzir texto sujo	Alta	Documentos mock gerados com estrutura controlada; extração determinística por ferramenta, não por LLM
Custo ou throttling do Bedrock	Média	Cadeia de fallback entre modelos; execuções manuais concentradas, com <code>LLM_PROVIDER=offline</code> restrito à suíte de testes
Avaliador ler o provider offline como fuga do requisito Bedrock	Alta se não mitigado	Bedrock como padrão sem fallback silencioso (ADR-05); acesso ao modelo solicitado no Dia 1; vídeo mostrando invocação real com <code>model_id</code> e tokens
Expansão de escopo	Alta	Seção “Fora” obrigatória em toda spec; SPEC-019 abandonável sem culpa
Documentação espremida no fim	Alta	SKILL.md escrito junto com cada agente, não no Dia 4; evidências coletadas durante a execução
Abstração especulativa acumulando ao longo da implementação	Média	Regra da seção 4.3 revisada a cada spec fechada: toda interface precisa de segunda implementação ou teste que a exija

8. Definição de pronto

O projeto está entregável quando, e apenas quando:

- `docker compose up -d` sobe tudo a partir de repositório limpo
- `make run` executa o pipeline de ponta a ponta e imprime `PipelineResult`
- `make test` fica verde sem rede e sem credenciais AWS
- `/docs` renderiza os três endpoints com exemplos
- Relatório JSON e PDF gerados e localizáveis
- Os 7 arquivos SKILL.md existem e seguem o formato
- README permite a um terceiro subir o projeto sem contato prévio
- Vídeo cobre scraping, análise, relatório e consulta à API, incluindo ao menos uma invocação real ao Bedrock visível em log
- Todos os 11 entregáveis da seção 5 do desafio estão mapeados no README
- As specs estão marcadas como `done` e refletem o que foi de fato construído
- Nenhuma classe abstrata ou protocolo sem implementação concreta no repositório; nenhum comentário que apenas parafraseia a linha seguinte